

**LAPORAN PRAKTIKUM**  
**POSTTEST 3**  
**ORGANISASI DAN ARSITEKTUR KOMPUTER**



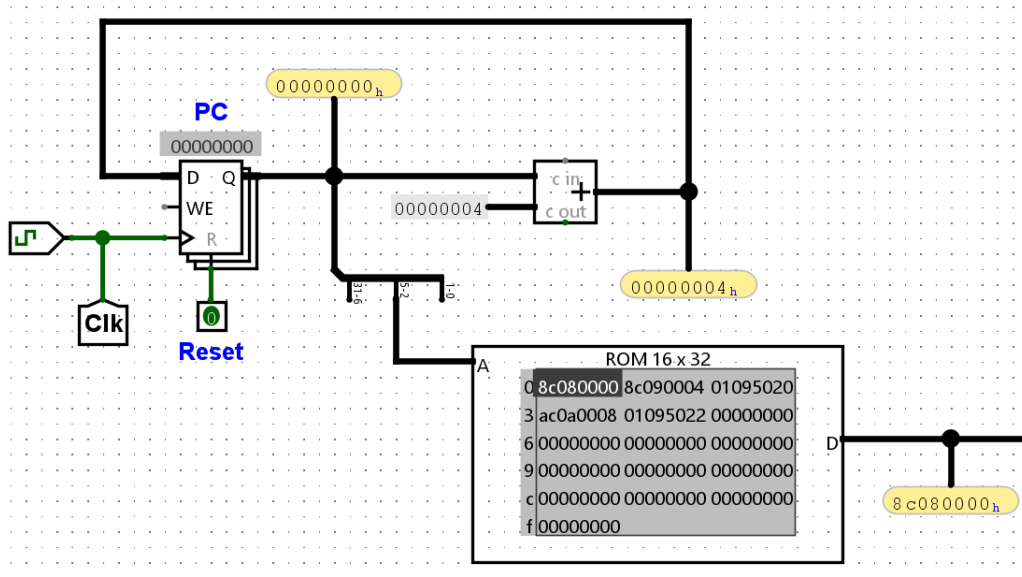
**Disusun oleh:**  
**Mauliyanda Yuan Hayfi (2609106071)**  
**Kelas (C1 '26)**

**PROGRAM STUDI INFORMATIKA**  
**UNIVERSITAS MULAWARMAN**  
**SAMARINDA**  
**2026**

# POSTTEST 3

## Organisasi & Arsitektur Komputer

### Soal 1: Unit Fetch (PC, Adder, dan Memori Instruksi)



Gambar 1.1

Rangkaian Unit Fetch di Logisim Evolution

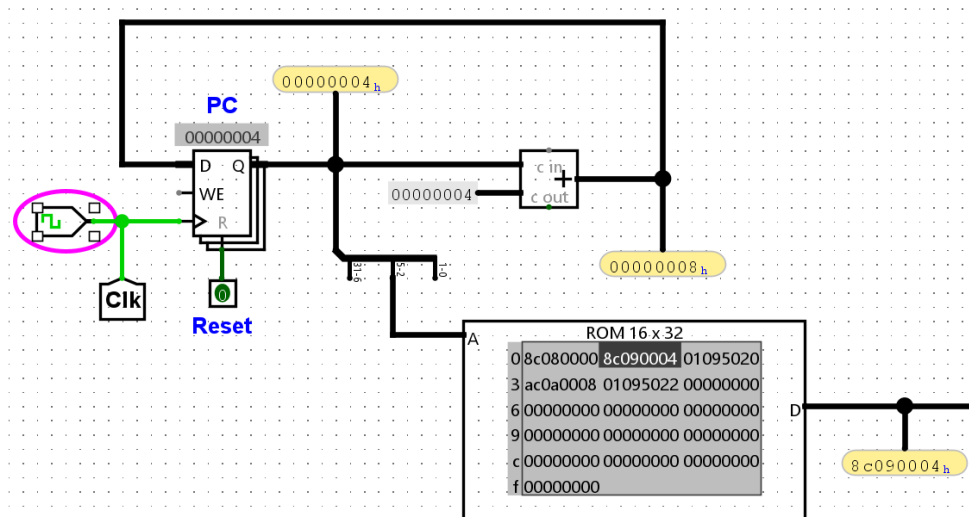
#### Cara Menguji:

1. Tekan **Clock** sebanyak dua kali, lalu amati perubahannya.
2. Ulangi terus hingga seluruh test-case terpenuhi.

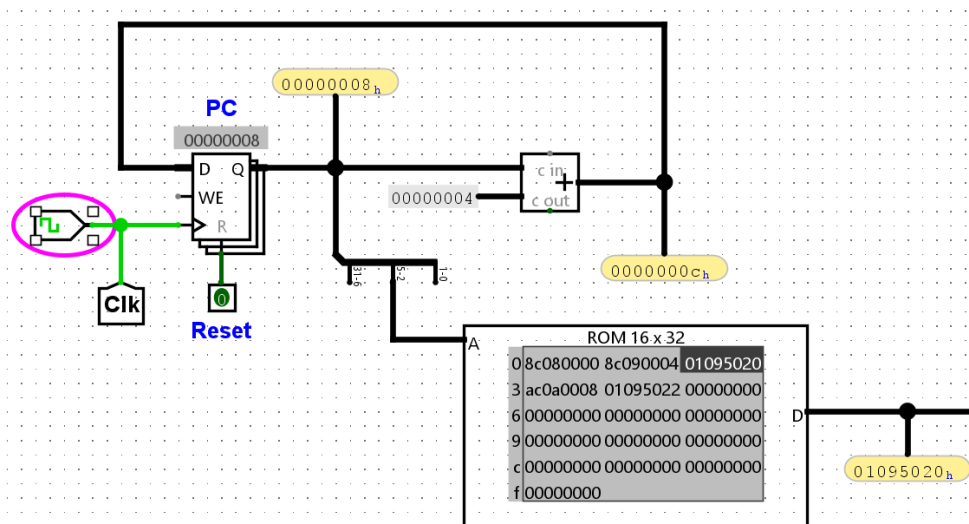
Tabel 1. Pengamatan unit fetch

| Tick ke- | PC (hex) | Alamat ROM (biner) | Instruksi (hex) | PC + 4 (hex) |
|----------|----------|--------------------|-----------------|--------------|
| 0        | 00000000 | 0000               | 8c080000        | 00000004     |
| 1        | 00000004 | 0001               | 8c090004        | 00000008     |
| 2        | 00000008 | 0010               | 01095020        | 0000000c     |
| 3        | 0000000c | 0011               | ac0a0008        | 00000010     |
| 4        | 00000010 | 0100               | 01095022        | 00000014     |
| 5        | 00000014 | 0101               | 00000000        | 00000018     |

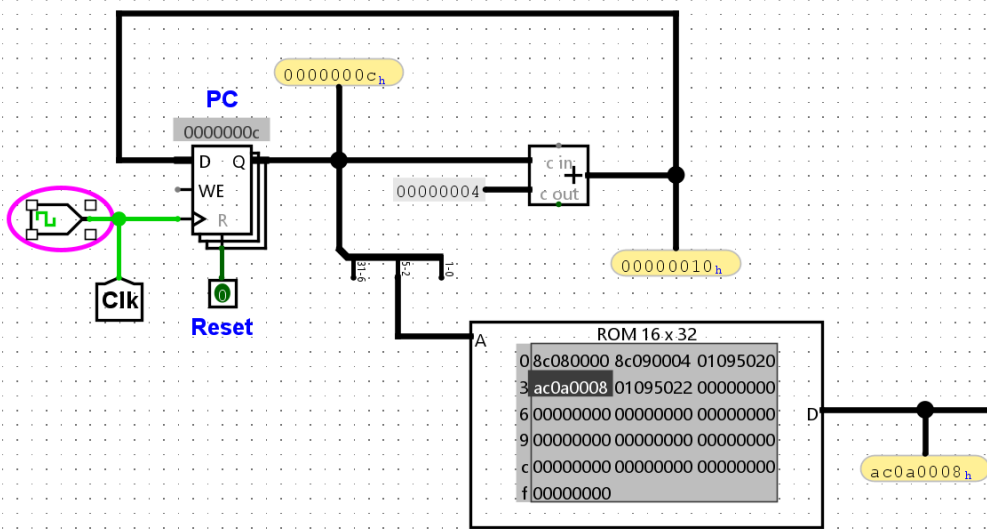
Satu tick adalah satu siklus hidup-mati clock. Alamat ROM menghitung dari 0 dan bertambah 1 setiap tick, kemudian dikonversikan dari desimal ke biner hingga menjadi sedemikian rupa.



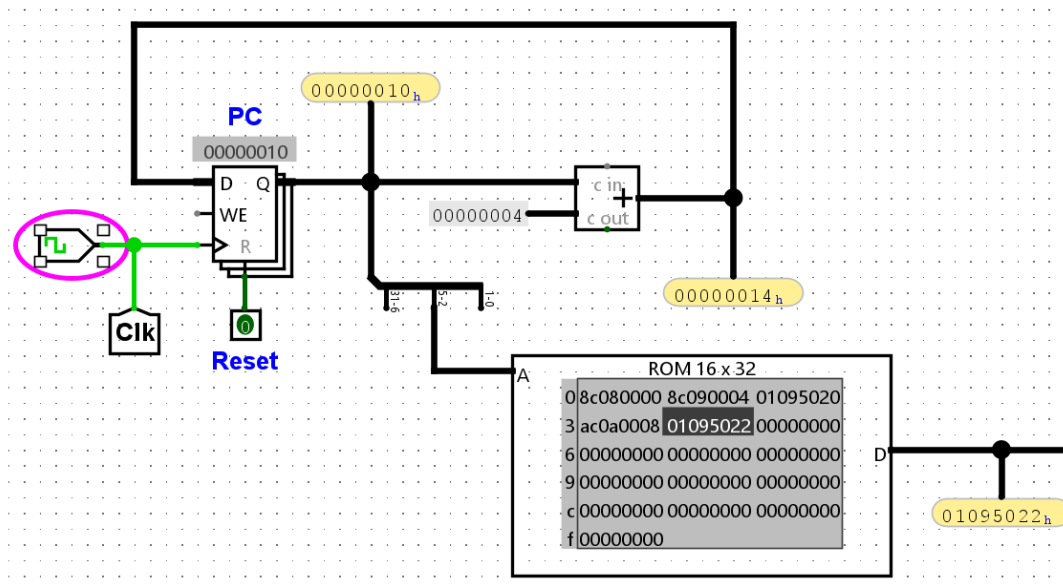
Gambar 1.2 Rangkaian fetch pada tick ke-1



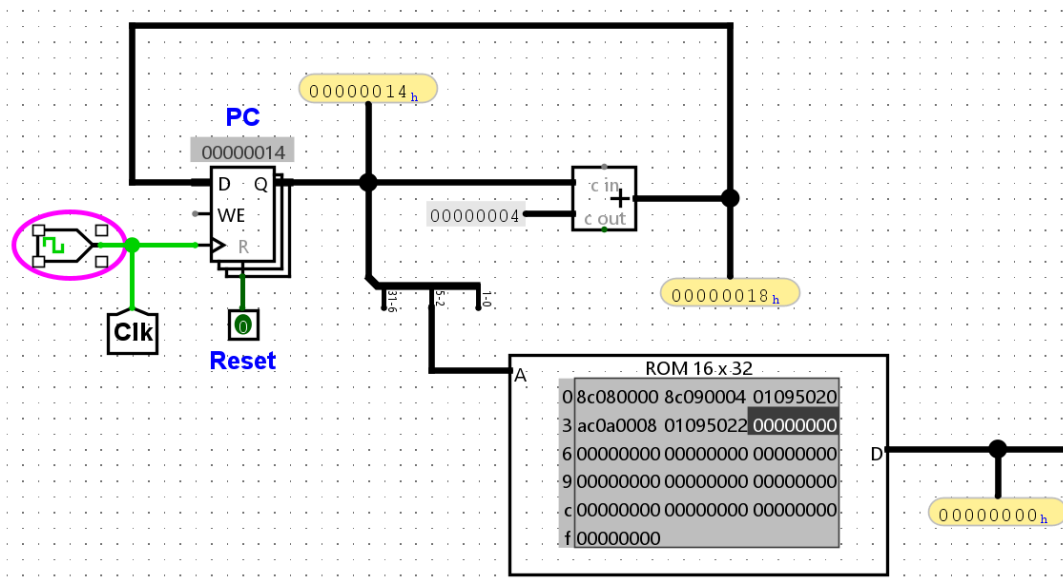
Gambar 1.3 Rangkaian fetch pada tick ke-2



Gambar 1.4 Rangkaian fetch pada tick ke-3



Gambar 1.5 Rangkaian fetch pada tick ke-4

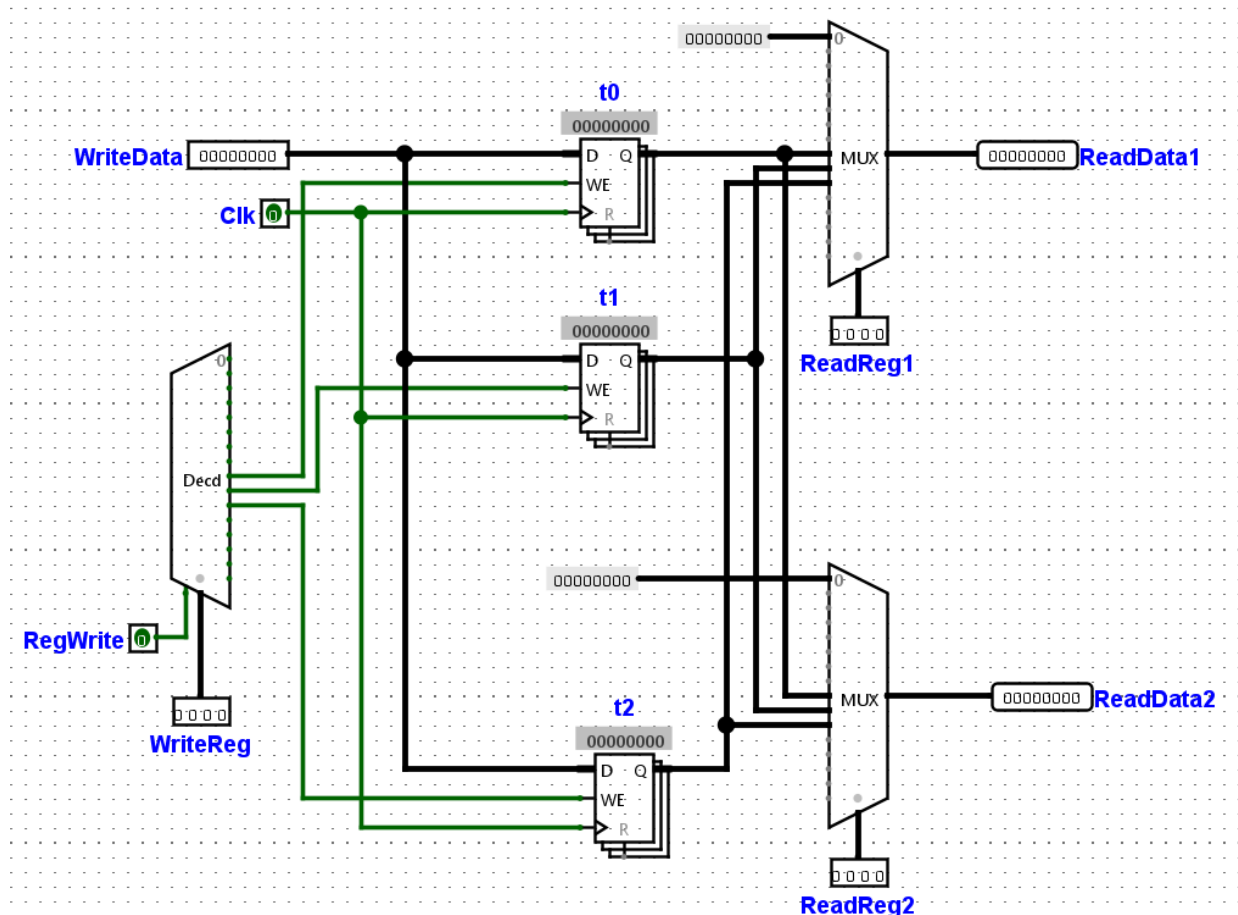


Gambar 1.6 Rangkaian fetch pada tick ke-5

**Keterangan:**

- Nilai PC (hex) diperoleh dari angka yang tertulis diatas **register**.
- Alamat ROM (biner) dimulai dari **0000** dan akan **terus bertambah satu** seiring **tick** berjalan.
- Nilai Instruksi (hex) didapat dari nilai **probe** yang tersambung dengan output **ROM**.
  - Nilai PC + 4 (hex) diperoleh dari hasil penjumlahan nilai **PC** dengan **bilangan 0x4**.

## Soal 2: Register File



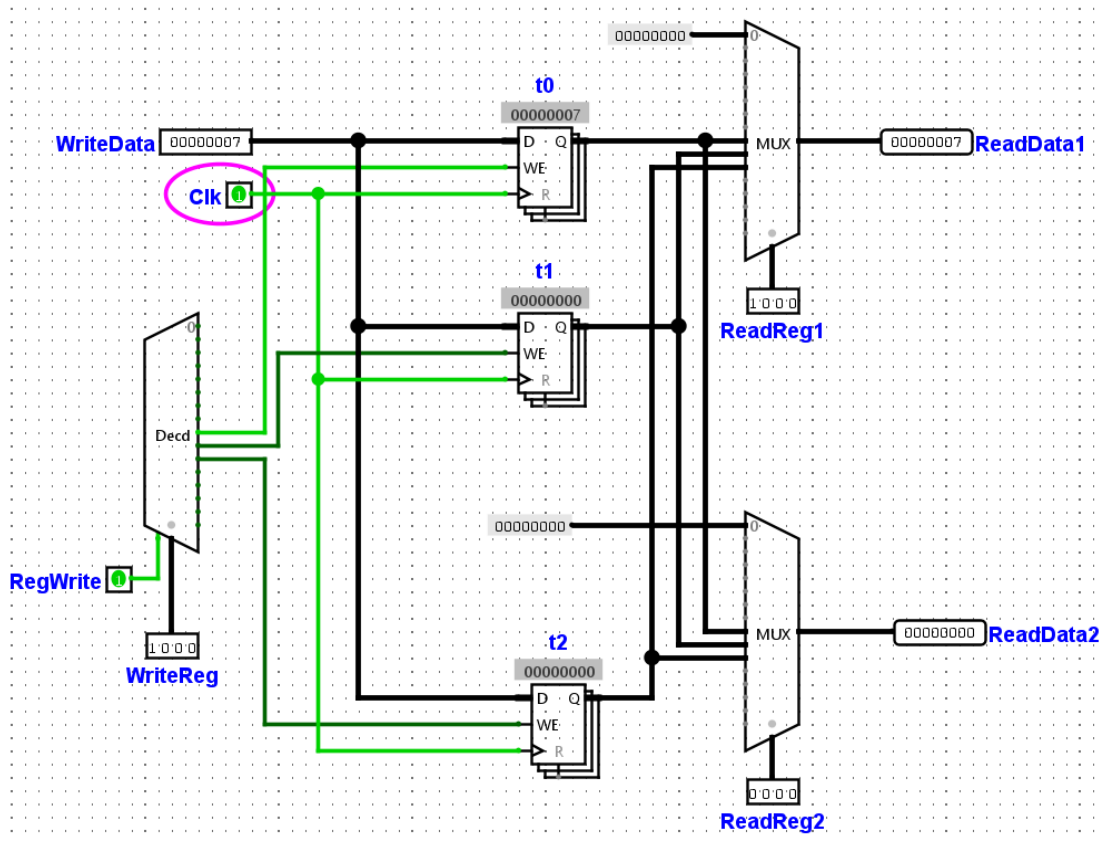
Gambar 2.1 Rangkaian Register File di Logisim Ecvolution

### Cara Menguji:

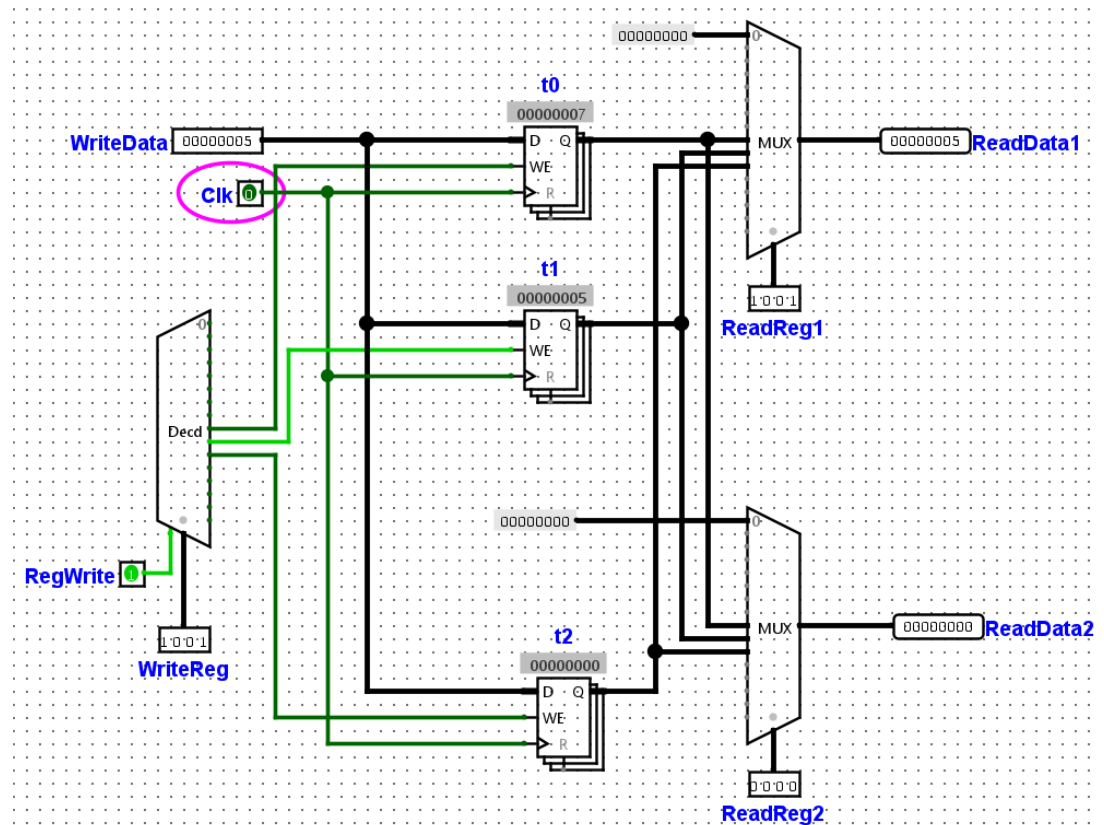
1. Isi semua bagian mulai dari **RegWrite**, **WriteReg**, dan **WriteData** terlebih dahulu. 2. Tekan **Clock** sebanyak dua kali.
3. Isi **ReadReg** sesuai dengan apa yang ada pada tabel, lalu amati perubahannya. 4. Ulangi terus hingga seluruh test-case terpenuhi.

Tabel 2.1 Pengujian register file 1

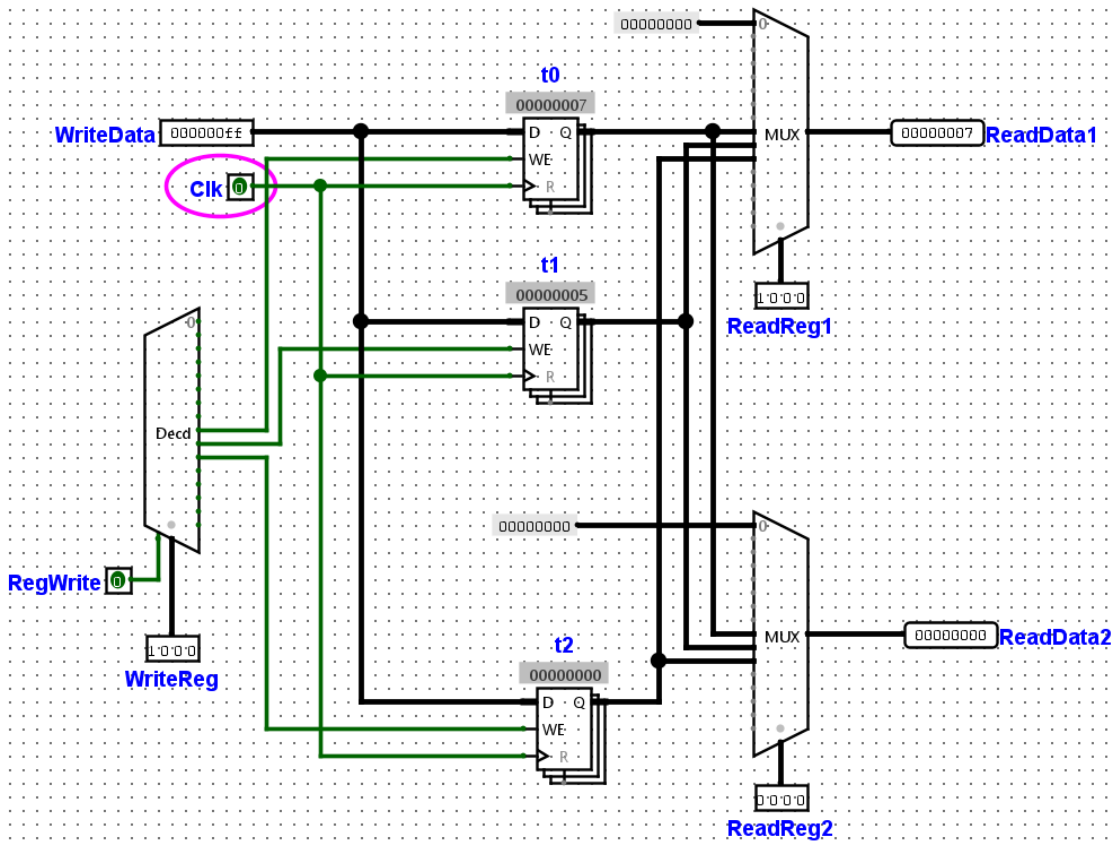
| No. | RegWrite | WriteReg | WriteData  | ReadReg1 | ReadData1 |
|-----|----------|----------|------------|----------|-----------|
| 1   | 1        | 1000     | 0x00000007 | 1000     | 00000007  |
| 2   | 1        | 1001     | 0x00000005 | 1001     | 00000005  |
| 3   | 0        | 1000     | 0x000000FF | 1000     | 00000007  |
| 4   | 1        | 0000     | 0x000000AA | 0000     | 00000000  |
| 5   | 1        | 1010     | 0x0000000C | 1010     | 0000000c  |



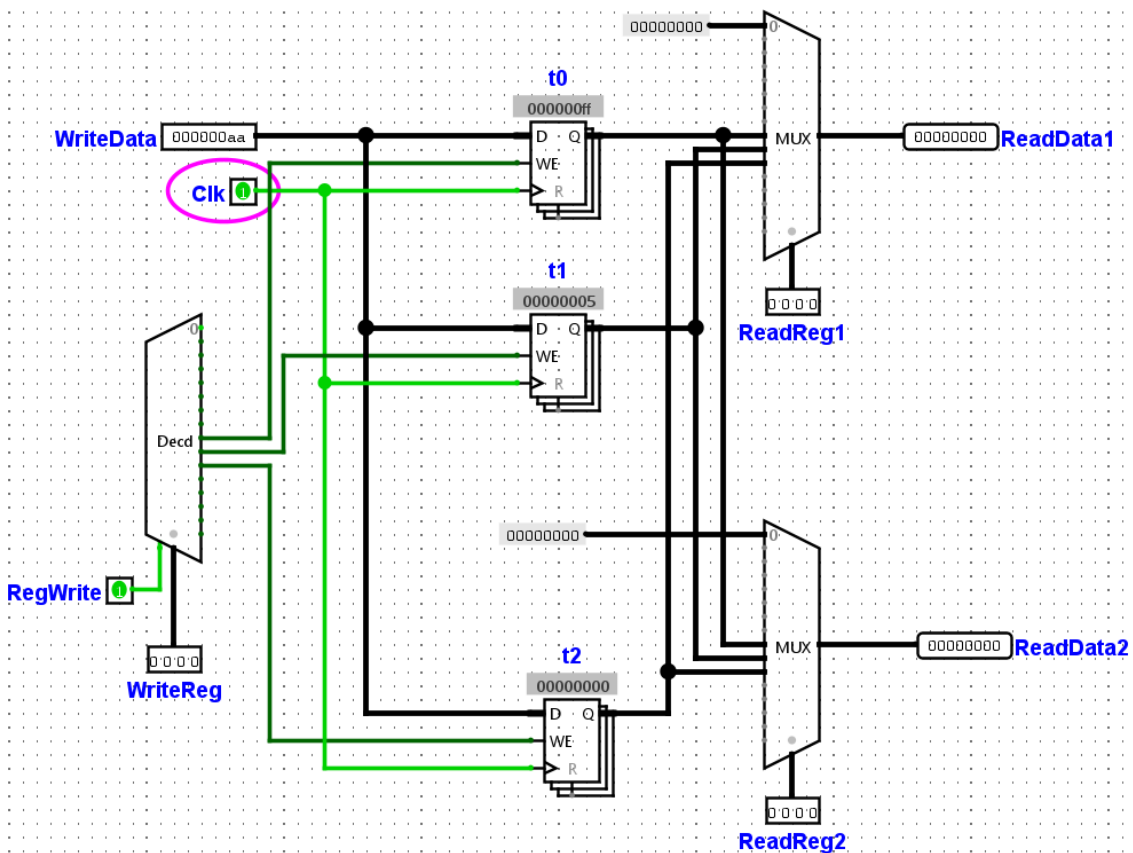
Gambar 2.2 Rangkaian Register File pada Test-case 1



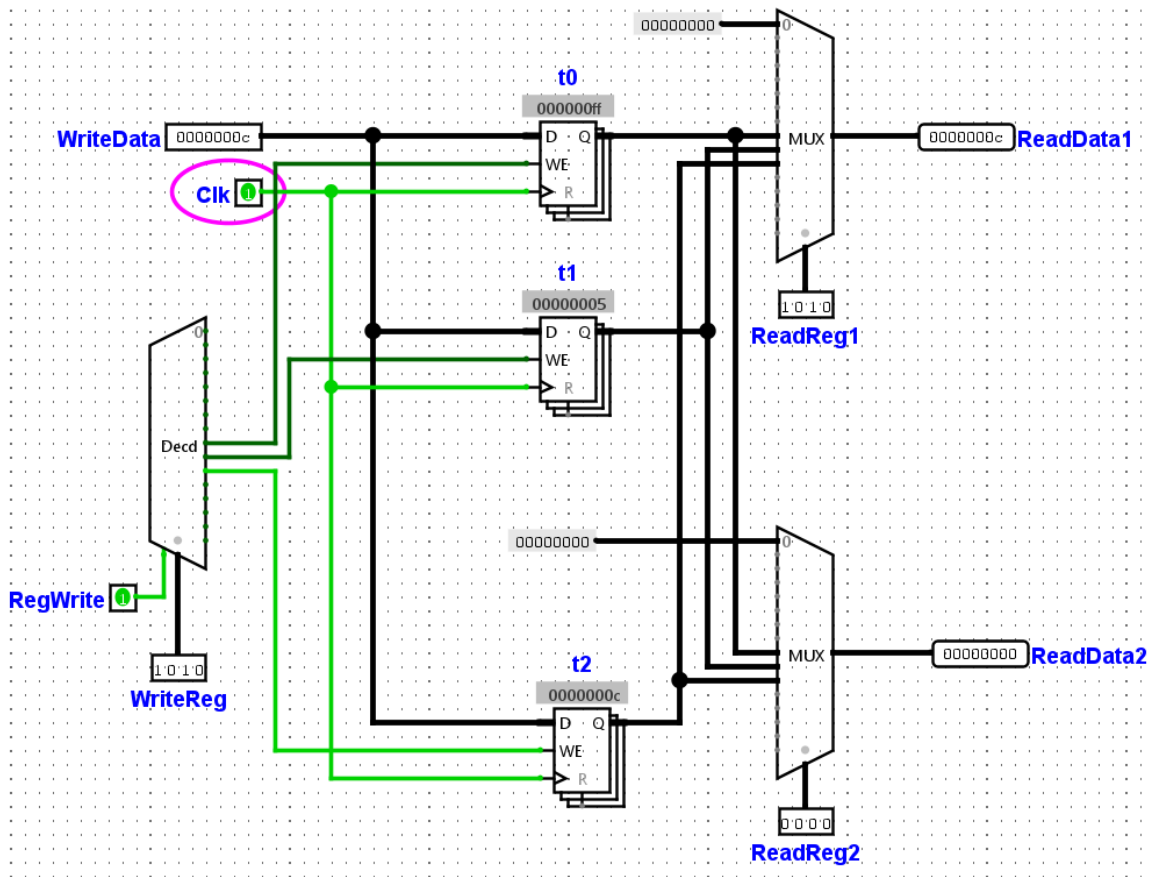
Gambar 2.3 Rangkaian Register File pada Test-case 2



Gambar 2.4 Rangkaian Register File pada Test-case 3



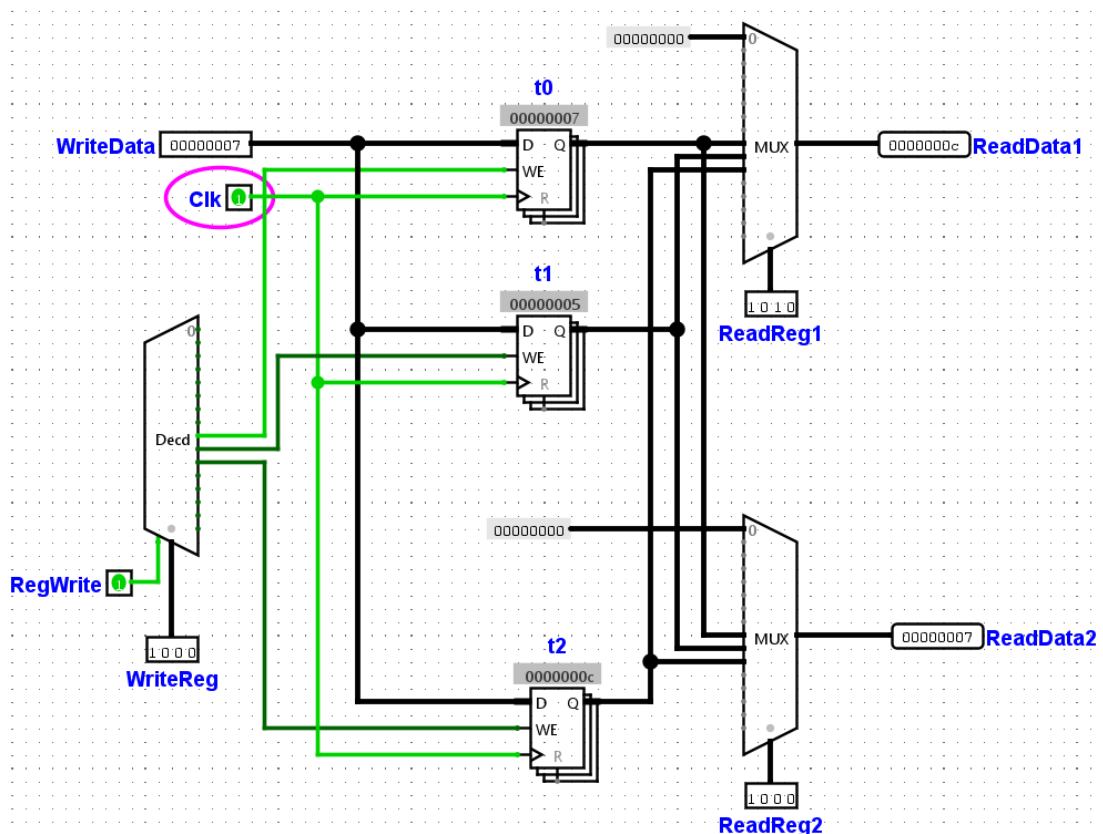
Gambar 2.5 Rangkaian Register File pada Test-case 4



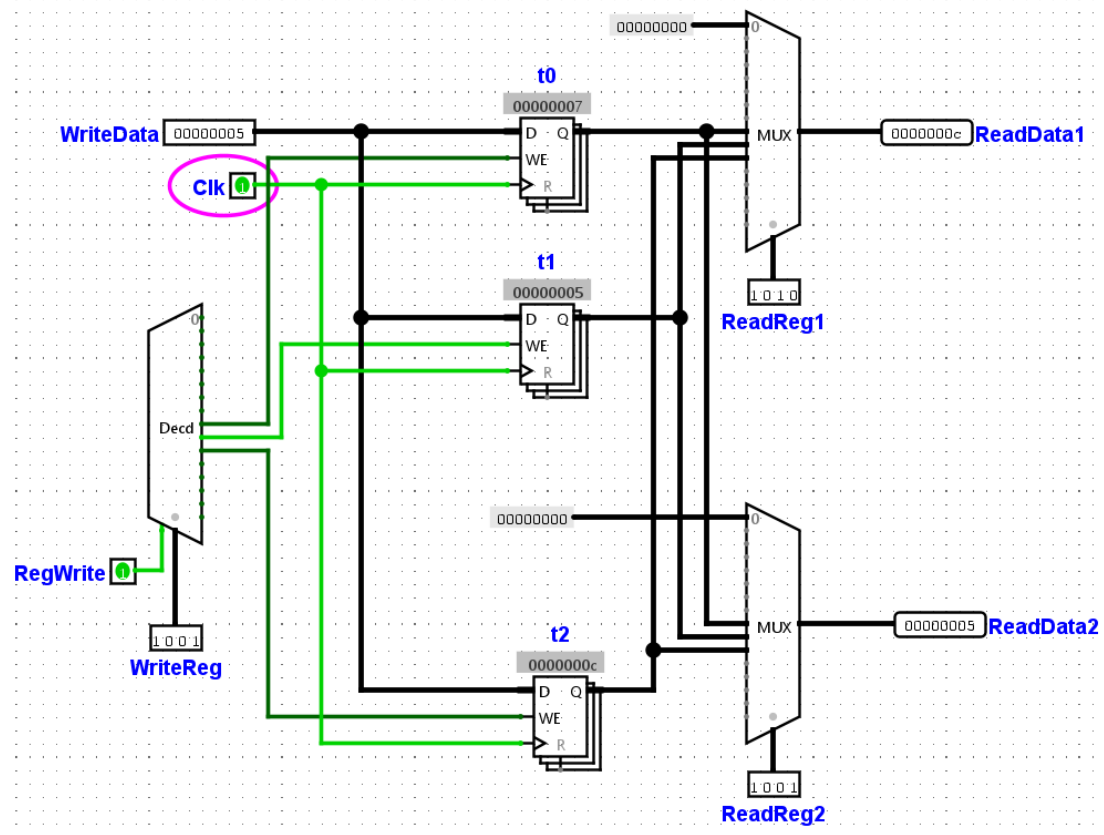
Gambar 2.6 Rangkaian Register File pada Test-case 5

Tabel 2.1 Pengujian register file 2

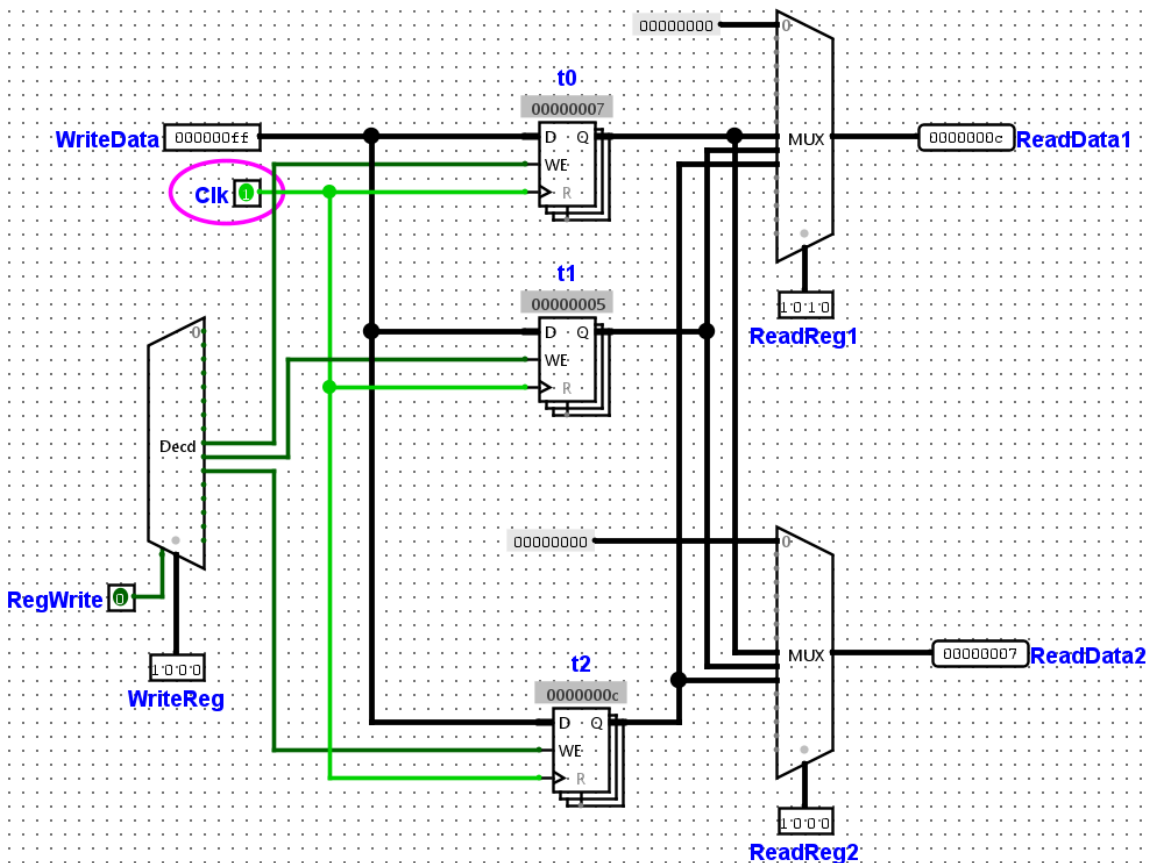
| No. | RegWrite | WriteReg | WriteData  | ReadReg2 | ReadData2 |
|-----|----------|----------|------------|----------|-----------|
| 1   | 1        | 1000     | 0x00000007 | 1000     | 00000007  |
| 2   | 1        | 1001     | 0x00000005 | 1001     | 00000005  |
| 3   | 0        | 1000     | 0x000000FF | 1000     | 00000007  |
| 4   | 1        | 0000     | 0x000000AA | 0000     | 00000000  |
| 5   | 1        | 1010     | 0x0000000C | 1010     | 0000000c  |



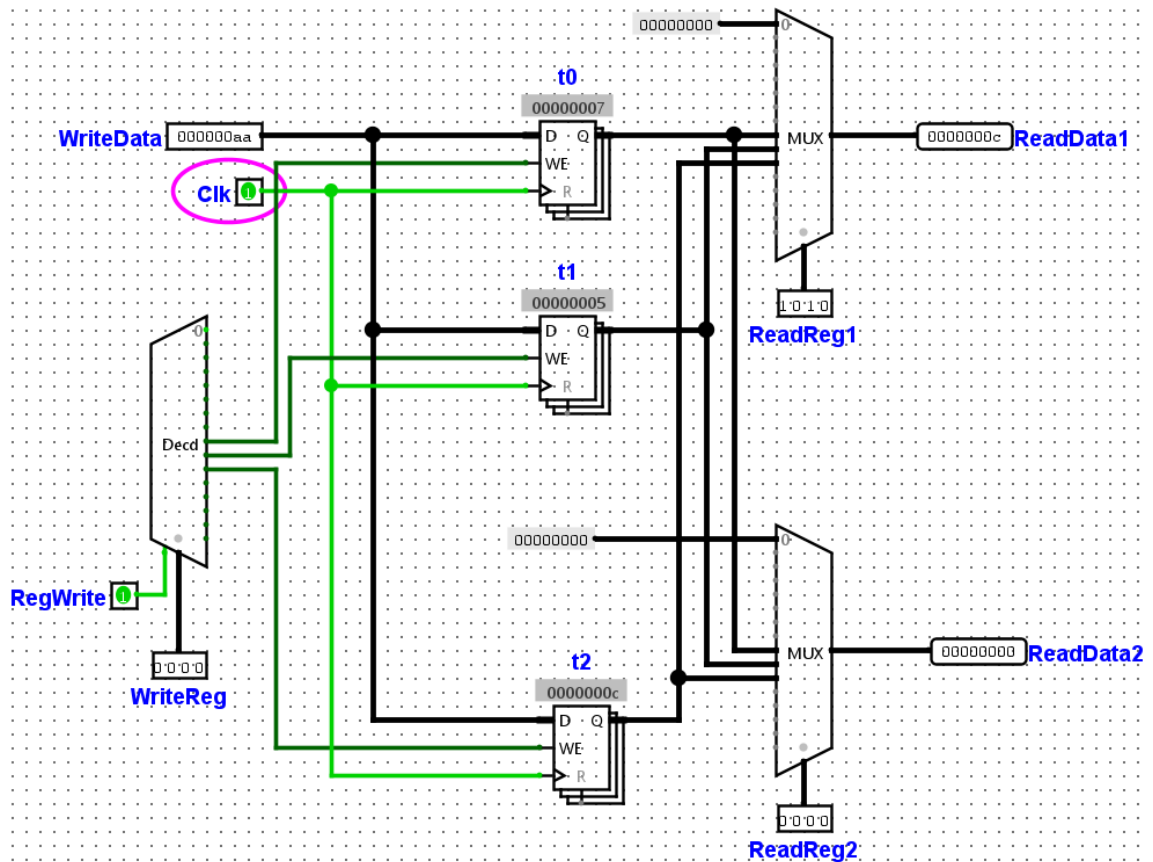
Gambar 2.7 Rangkaian Register File pada Test-case 6



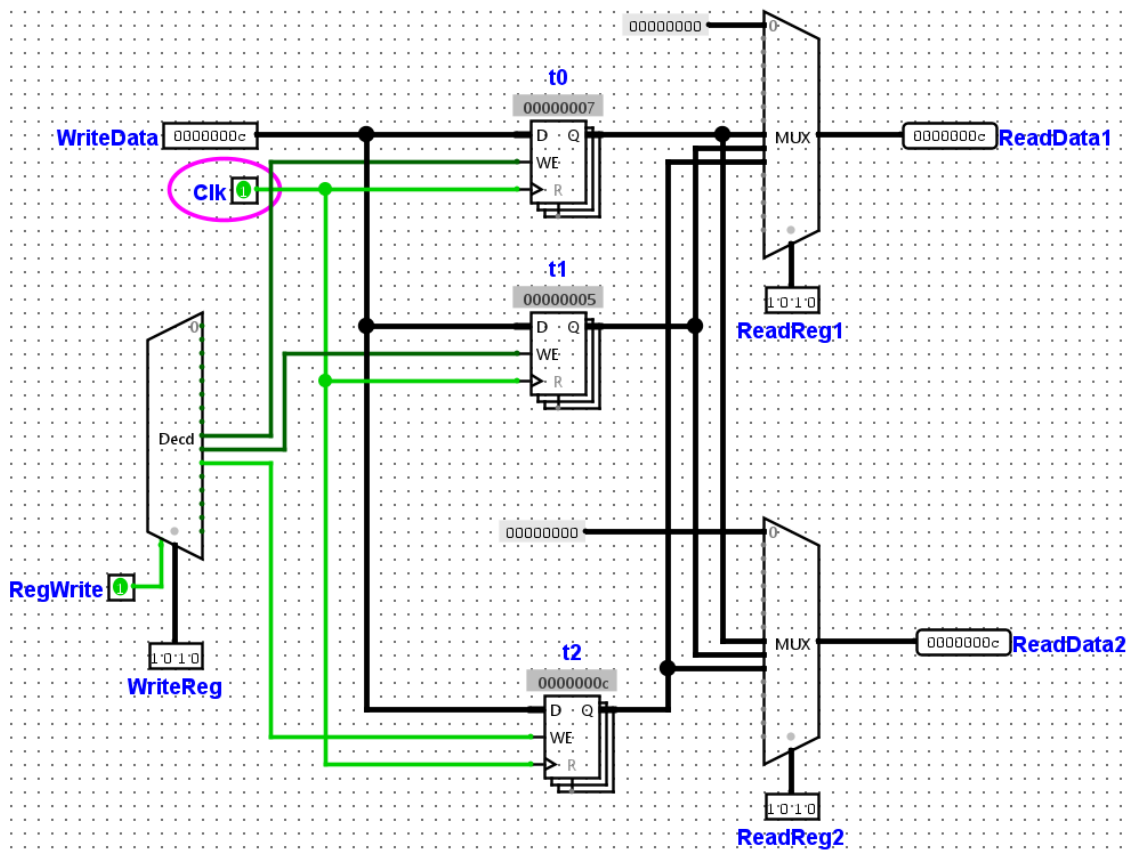
Gambar 2.8 Rangkaian Register File pada Test-case 7



Gambar 2.9 Rangkaian Register File pada Test-case 8

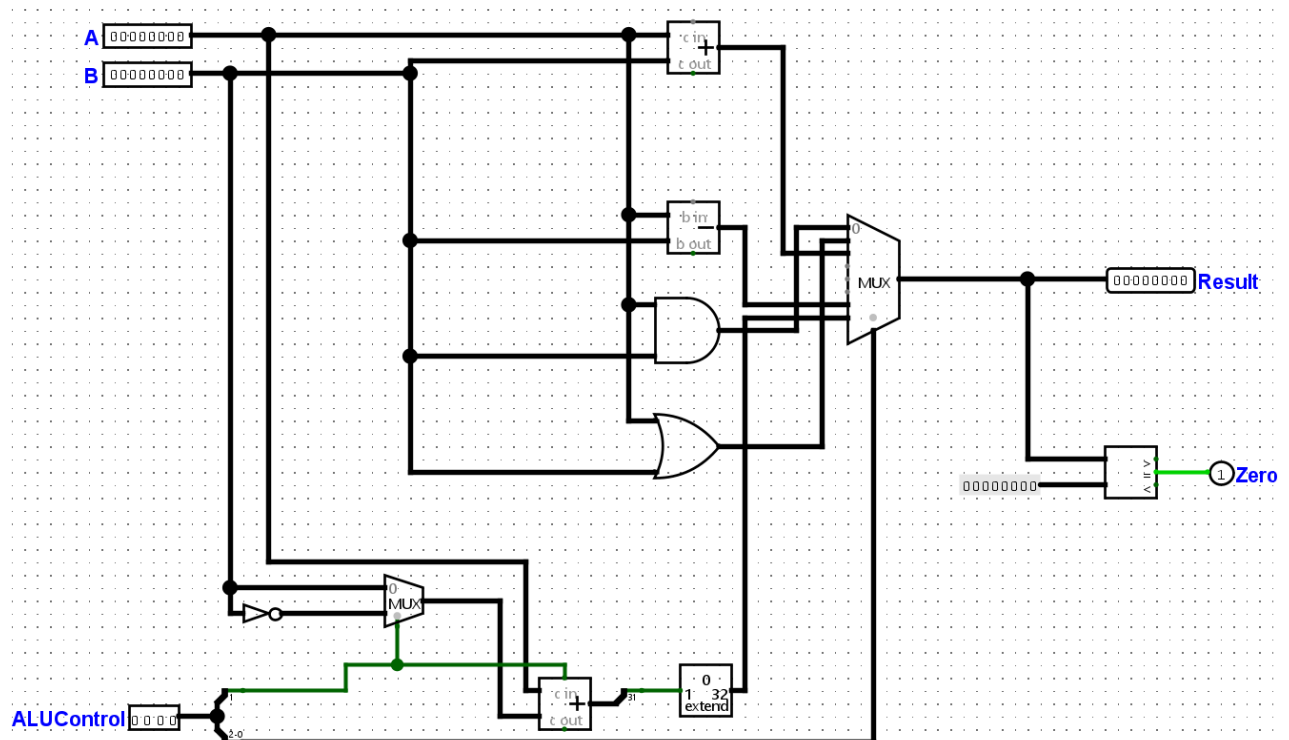


Gambar 2.10 Rangkaian Register File pada Test-case 9



Gambar 2.11 Rangkaian Register File pada Test-case 10

### Soal 3: ALU



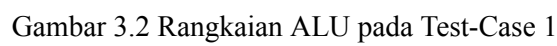
Gambar 3.1 Rangkaian ALU di Logisim Evolusion

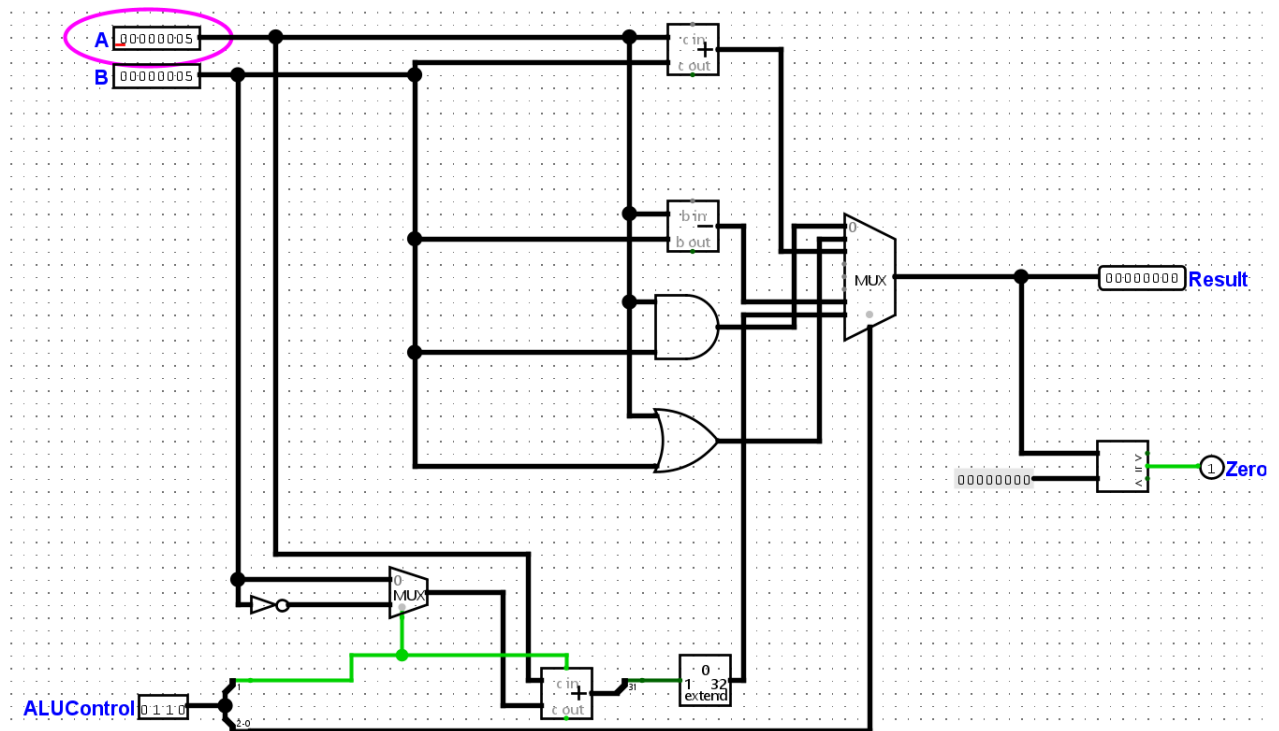
#### Cara Menguji:

1. Masukkan nilai **A** dan **B**. Kemudian masukkan nilai dari **ALU Control**. Kemudian amati perubahannya.
2. Ulangi terus hingga seluruh test-case terpenuhi.

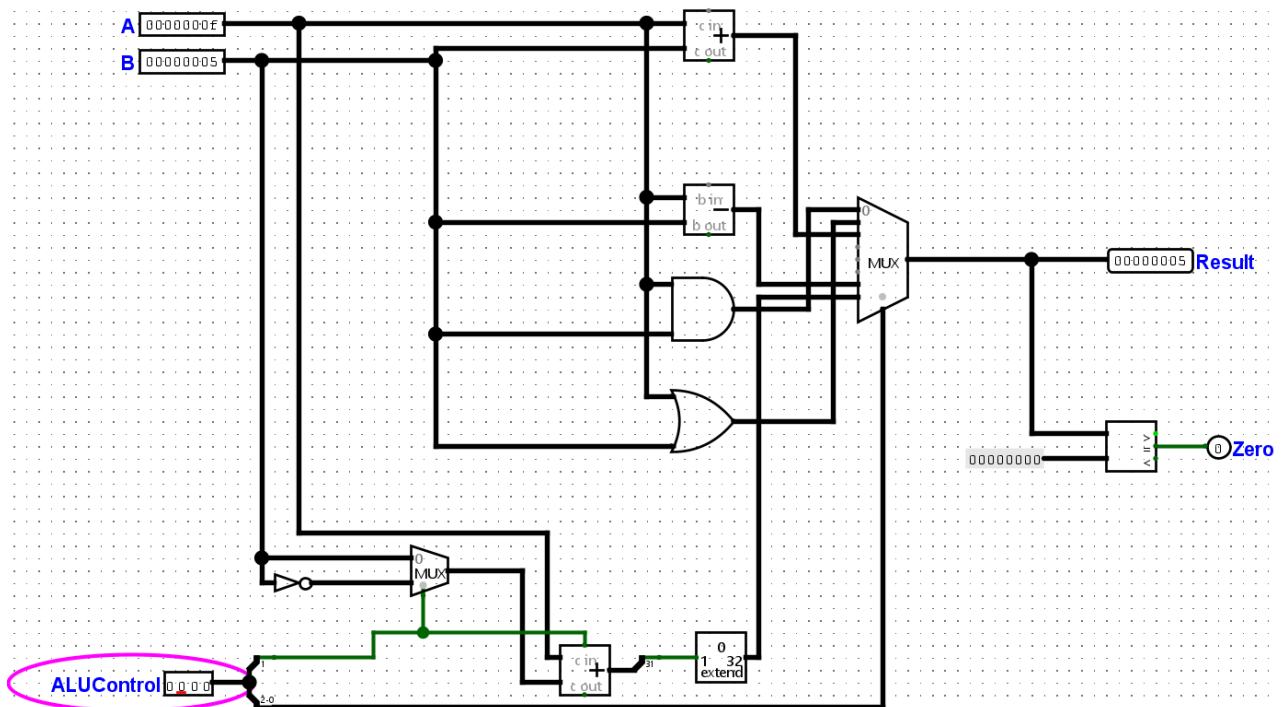
Tabel 14. Pengujian ALU

| No. | A (hex)    | B (hex)    | ALU Control | Operasi | Result   | Zero |
|-----|------------|------------|-------------|---------|----------|------|
| 1   | 0x00000007 | 0x00000005 | 0010        | add     | 0000000c | 0    |
| 2   | 0x00000007 | 0x00000005 | 0110        | sub     | 00000002 | 0    |
| 3   | 0x00000005 | 0x00000005 | 0110        | sub     | 00000000 | 1    |
| 4   | 0x0000000F | 0x00000005 | 0000        | AND     | 00000005 | 0    |
| 5   | 0x0000000F | 0x00000005 | 0001        | OR      | 0000000f | 0    |
| 6   | 0x00000003 | 0x00000009 | 0111        | slt     | 00000001 | 0    |

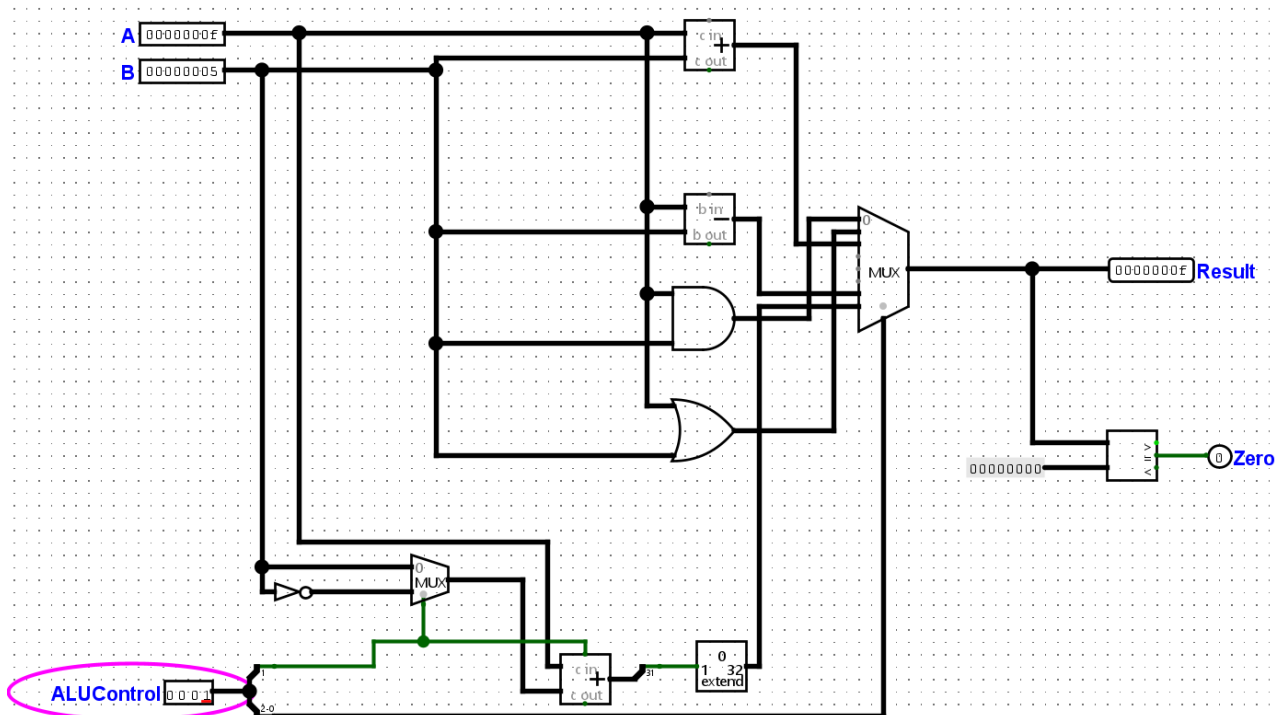




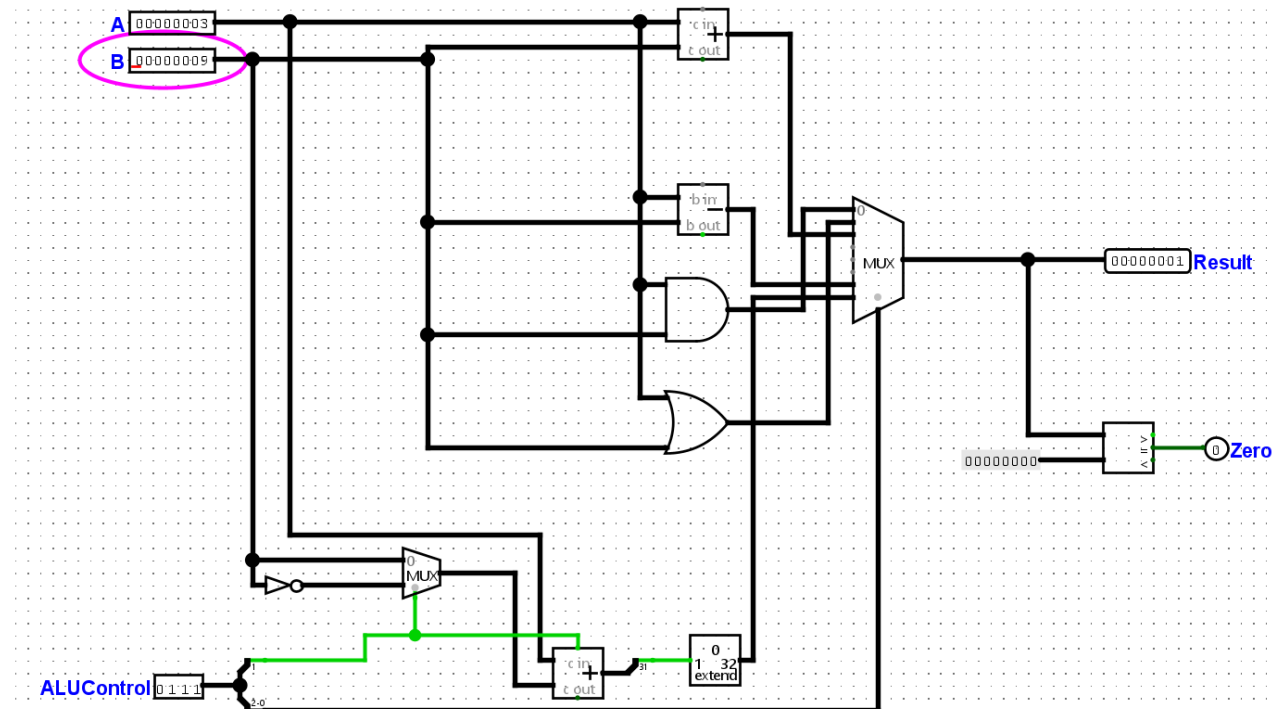
Gambar 3.4 Rangkaian ALU pada Test-Case 3



Gambar 3.5 Rangkaian ALU pada Test-Case 4

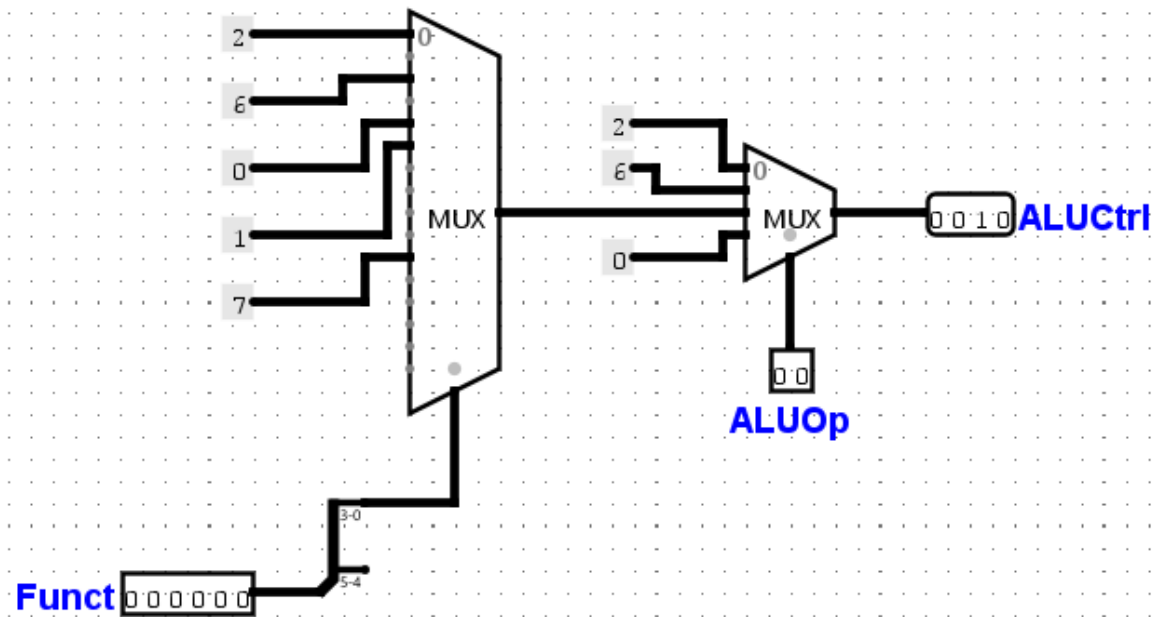


Gambar 3.6 Rangkaian ALU pada Test-Case 5



Gambar 3.7 Rangkaian ALU pada Test-Case 6

#### Soal 4: ALU Control



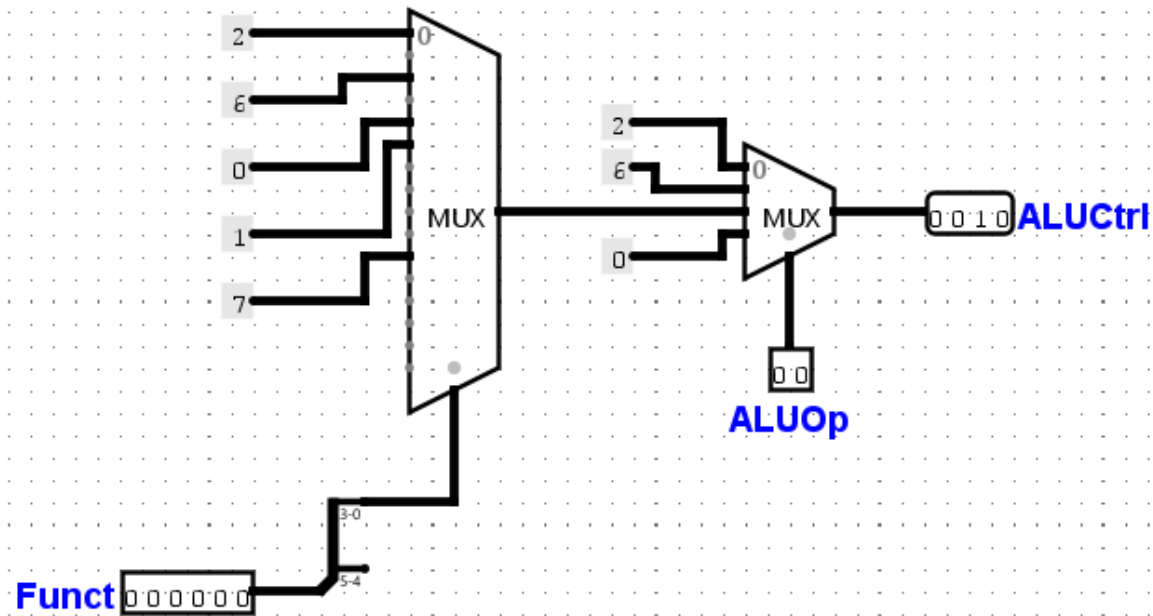
Gambar 4.1 Rangkaian ALU Control pada Logisim Evolution

#### Cara Menguji:

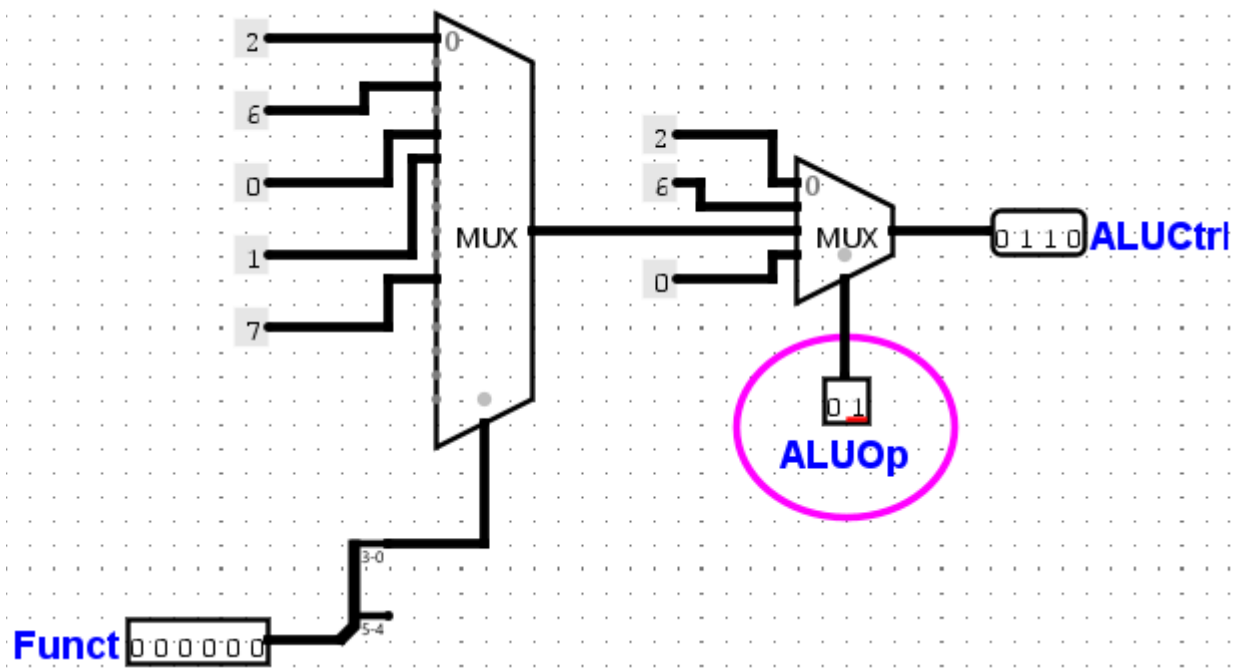
1. Masukkan nilai **ALUOp**, kemudian dilanjutkan oleh **Funct**, lalu amati perubahannya.
2. Ulangi terus hingga seluruh test-case terpenuhi.

Tabel 15. Pengujian ALU control

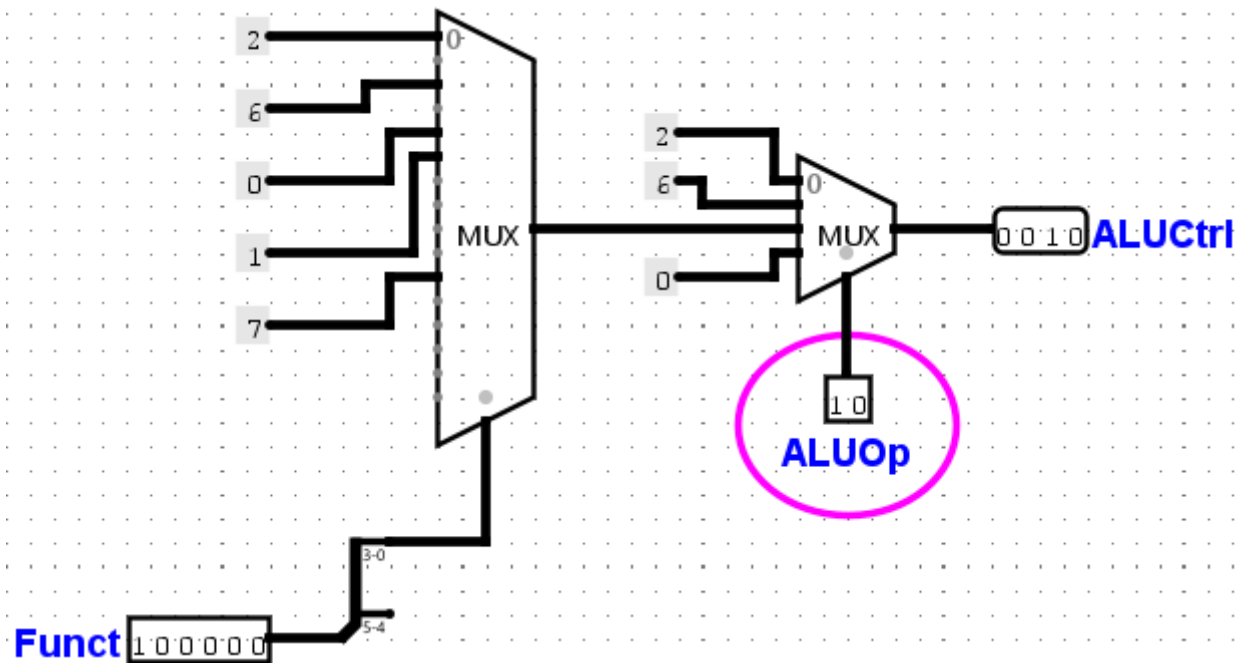
| No. | Instruksi | ALUOp | Funct  | ALUCtrl | Operasi ALU                                                                 |
|-----|-----------|-------|--------|---------|-----------------------------------------------------------------------------|
| 1   | lw / sw   | 00    | XXXXXX | 0010    | Menghitung alamat memori (Penjumlahan / Add)                                |
| 2   | beq       | 01    | XXXXXX | 0110    | Membandingkan kedua nilai dari dua Source Register.                         |
| 3   | add       | 10    | 100000 | 0010    | Membandingkan kedua nilai dari dua Source Register (Pengurangan / Subtract) |
| 4   | sub       | 10    | 100010 | 0110    | Penjumlahan (Add)                                                           |
| 5   | and       | 10    | 100100 | 0000    | Pengurangan (Subtract)                                                      |
| 6   | or        | 10    | 100101 | 0001    | Operasi logika AND                                                          |
| 7   | slt       | 10    | 101010 | 0111    | Operasi logika OR                                                           |



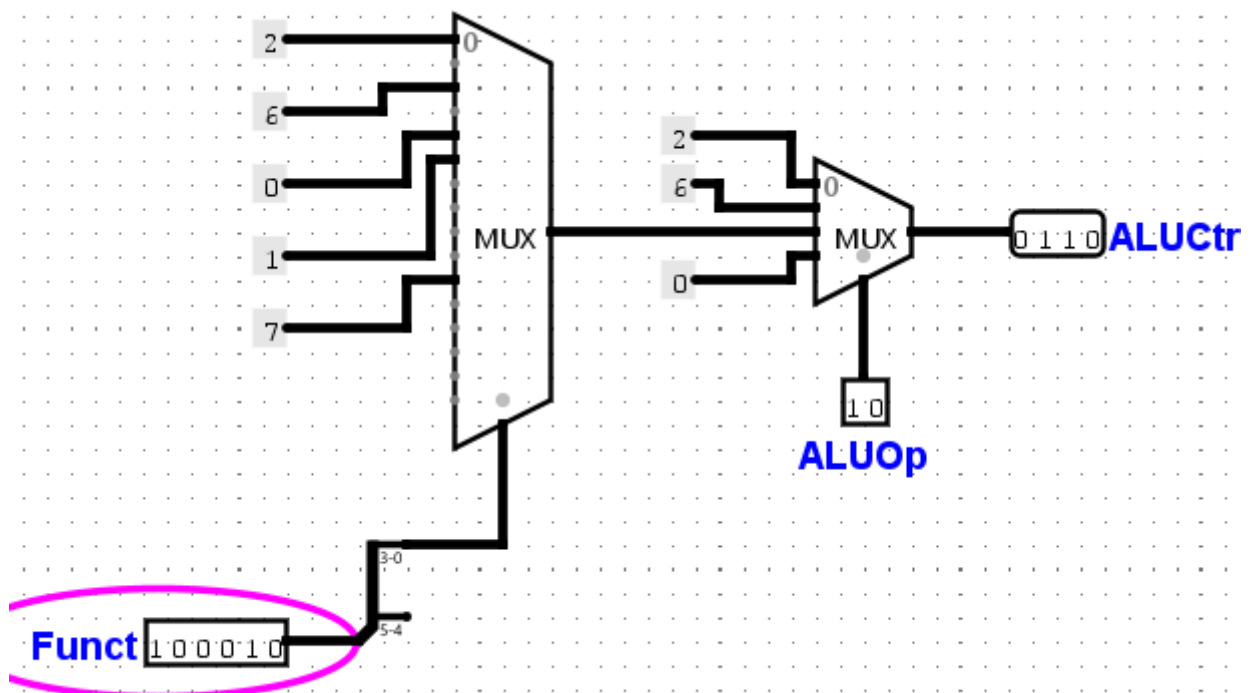
Gambar 4.2 Rangkaian ALU Control pada Test-Case 1



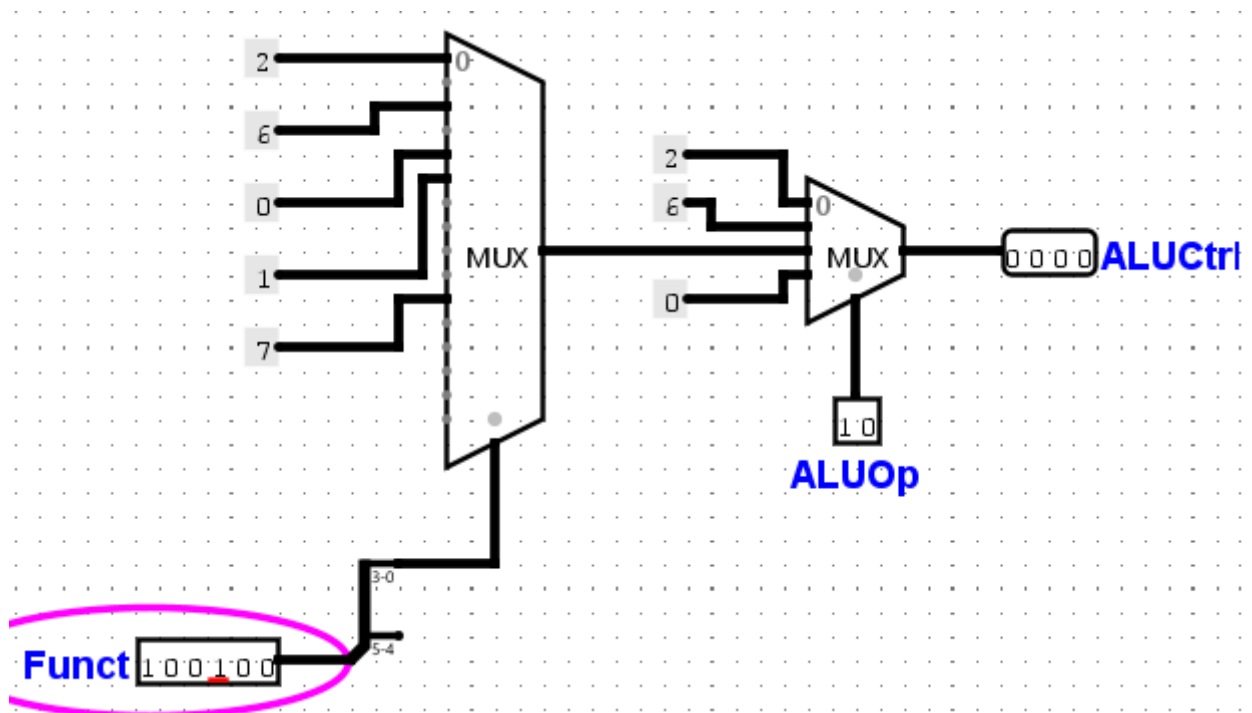
Gambar 4.3 Rangkaian ALU Control pada Test-Case 2



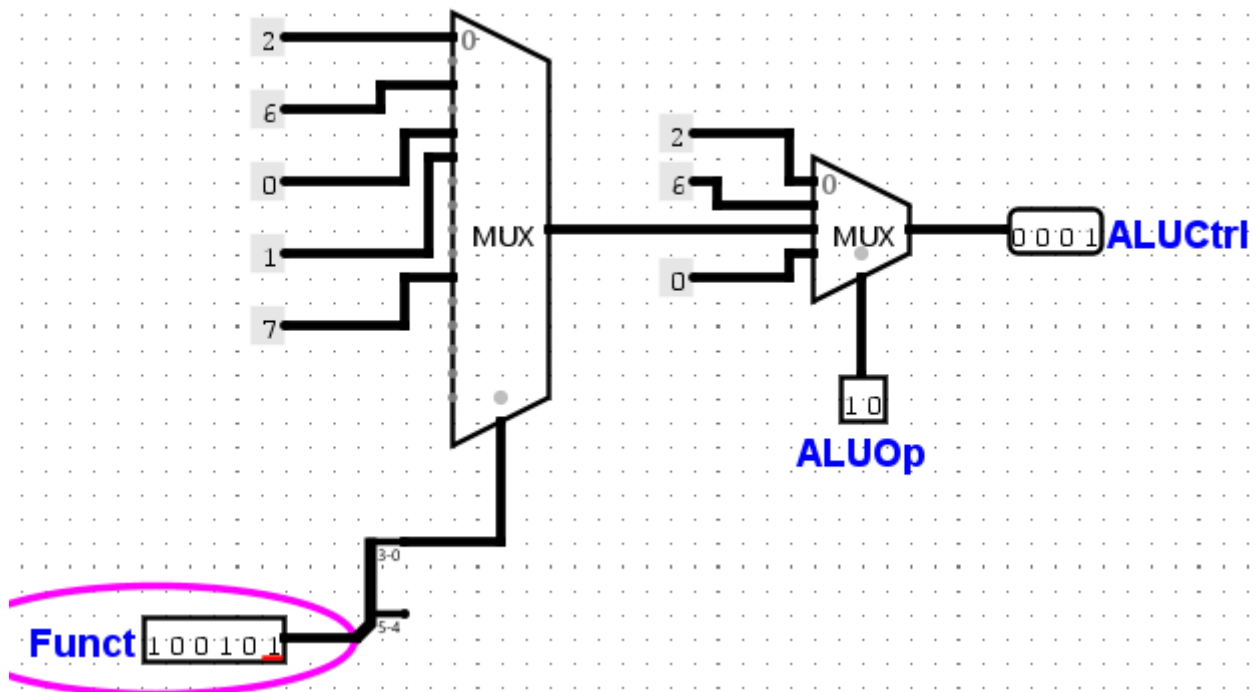
Gambar 4.4 Rangkaian ALU Control pada Test-Case 3



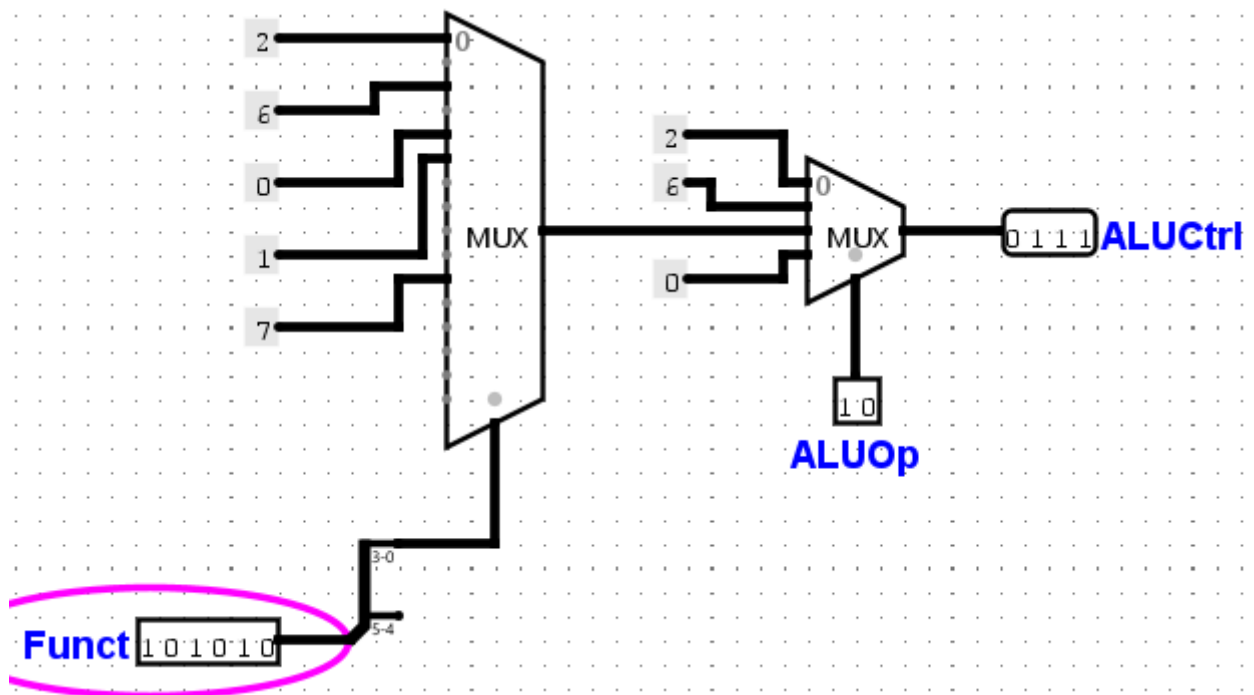
Gambar 4.5 Rangkaian ALU Control pada Test-Case 4



Gambar 4.6 Rangkaian ALU Control pada Test-Case 5

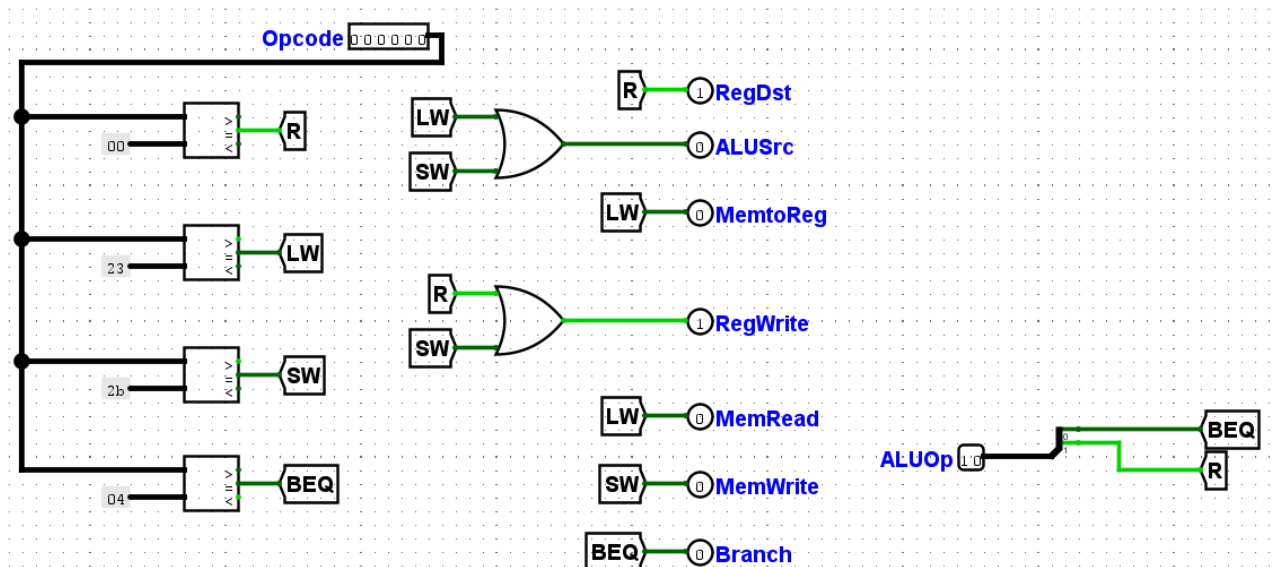


Gambar 4.7 Rangkaian ALU Control pada Test-Case 6



Gambar 4.8 Rangkaian ALU Control pada Test-Case 7

### Soal 5: Control Unit

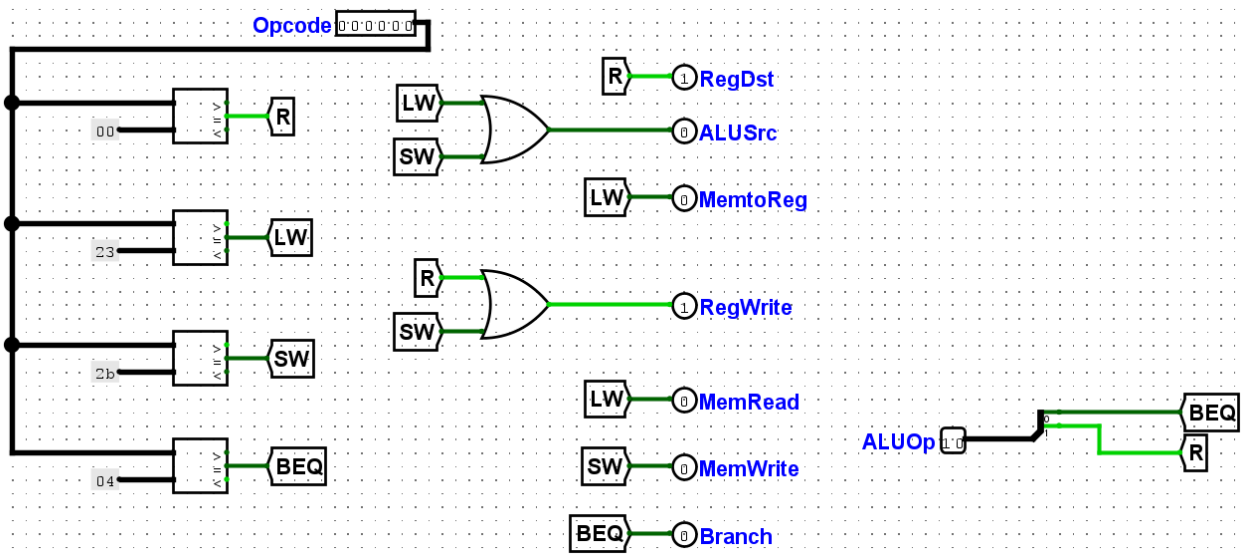


#### Cara Menguji:

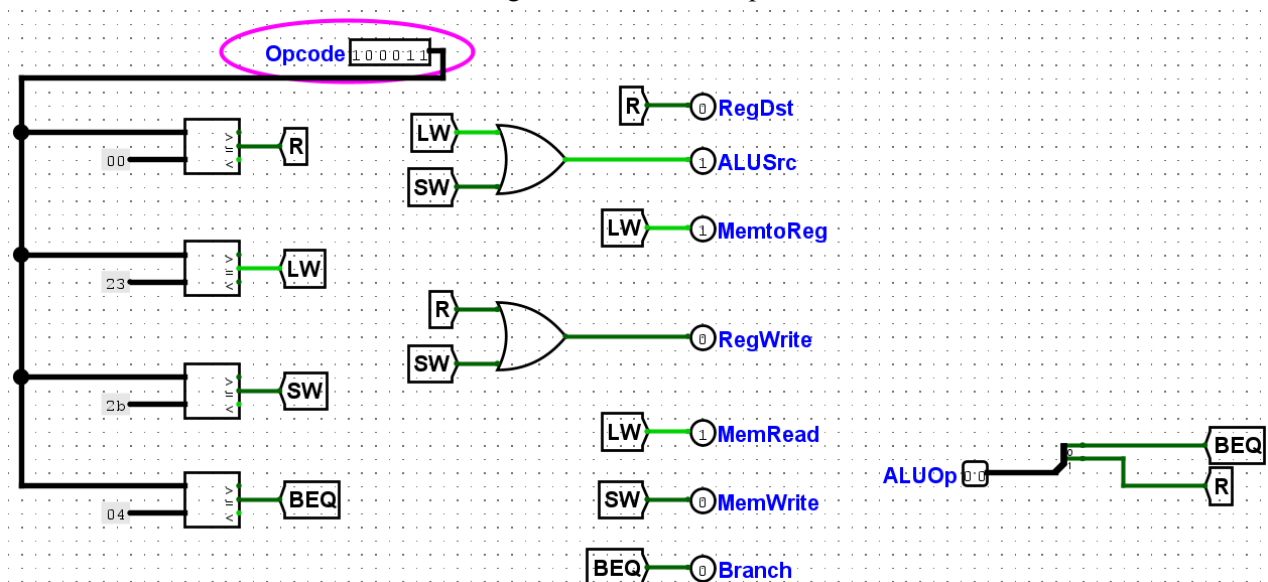
1. Isi **Opcode** sesuai dengan tabel yang ada, lalu amati perubahannya.
2. Ulangi terus hingga seluruh test-case terpenuhi.

Tabel 16. Pengujian control unit

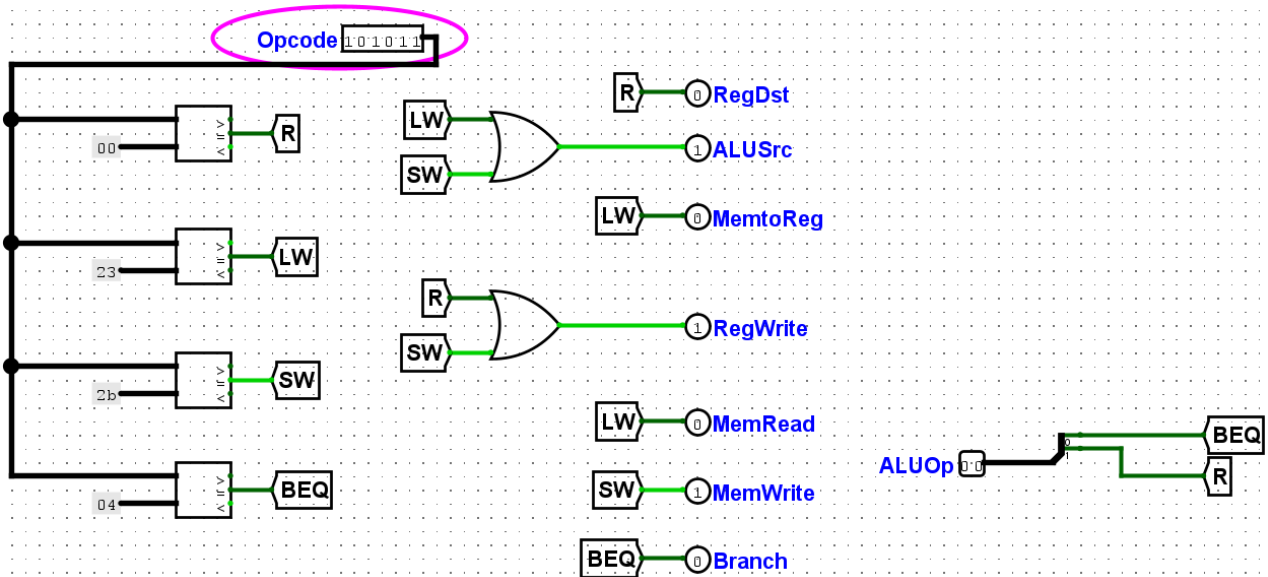
| Instruksi (opcode) | Reg Dst | ALU Src | Memto Reg | Reg Write | Mem Read | Mem Write | Branch | ALUOp |
|--------------------|---------|---------|-----------|-----------|----------|-----------|--------|-------|
| R-type (000000)    | 1       | 0       | 0         | 1         | 0        | 0         | 0      | 10    |
| lw (100011)        | 0       | 1       | 1         | 0         | 1        | 0         | 0      | 00    |
| sw (101011)        | 0       | 1       | 0         | 1         | 0        | 1         | 0      | 00    |
| beq (000100)       | 0       | 0       | 0         | 0         | 0        | 0         | 1      | 01    |



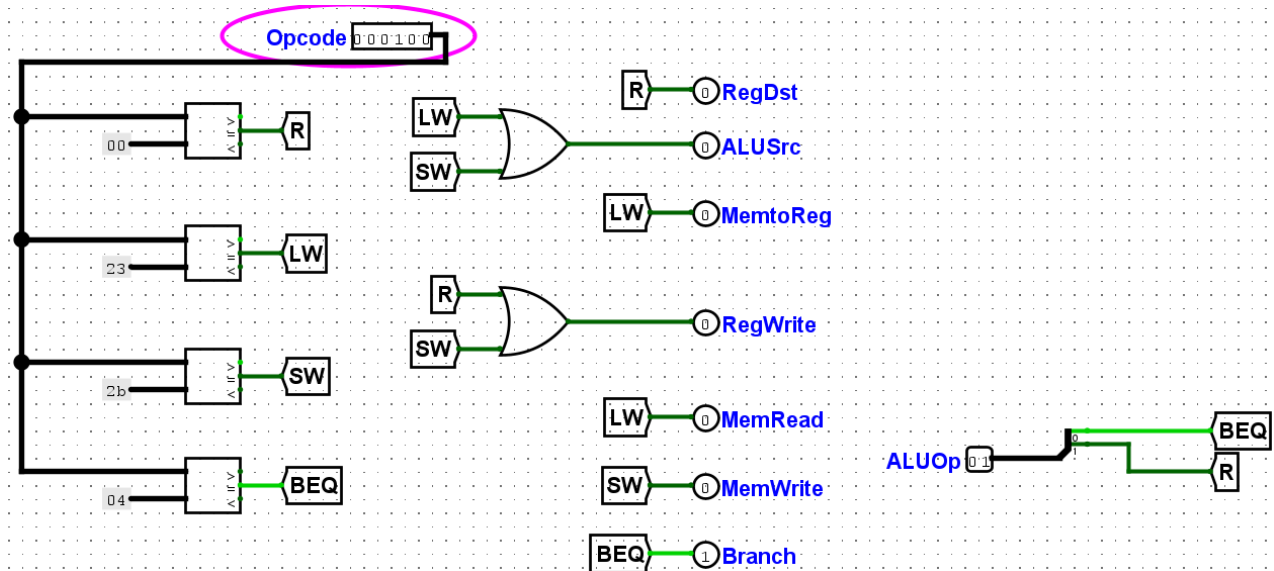
Gambar 5.2 Rangkaian Control Unit pada Test-Case 1



Gambar 5.3 Rangkaian Control Unit pada Test-Case 2



Gambar 5.4 Rangkaian Control Unit pada Test-Case 3



Gambar 5.5 Rangkaian Control Unit pada Test-Case 4